

Übungsblatt: Algorithmic Graphics Generation

Aufgabe 1 – Vertikaler Gradient

Implementiere einen vertikalen Grauwertverlauf.

Die Intensität jedes Pixels soll berechnet werden durch:

$$I(x,y)=y$$

Anforderungen

1. Erzeuge ein Bild der Größe:
 - 512×512 Pixel
2. Berechne den Intensitätswert jedes Pixels.
3. Normalisiere die Werte auf den Bereich:
 - 0–255
4. Speichere das Ergebnis als PNG-Datei.

Hinweise

- Verwende ein NumPy-Array.
 - Arbeite pixelweise.
 - Visualisiere das Ergebnis mit Matplotlib.
-

Aufgabe 2 – Periodisches Muster

Implementiere folgendes Muster:

$$I(x,y)=\sin(y)$$

Anforderungen

1. Erzeuge ein periodisches Streifenmuster.
2. Skaliere die Sinuswerte sinnvoll auf:
 - 0–255
3. Experimentiere mit verschiedenen Frequenzen.

Zusatzaufgabe

Verändere die Funktion zu:

$$I(x,y)=\sin(x)\cdot\cos(y)$$

Wie verändert sich das Muster?

Aufgabe 3 – White Noise

Generiere ein White-Noise-Bild.

Anforderungen

1. Erzeuge ein Graustufenbild:
 - 512×512 Pixel
2. Jeder Pixel soll einen zufälligen Wert besitzen:
 - 0–255
3. Verwende:
 - gleichverteilte Zufallszahlen (nutze z.B. `np.random.randint(0, 256)`)
4. Speichere das Bild.

Zusatzaufgabe

Wie verändert sich das Bild, wenn statt gleichverteilter Zufallszahlen normalverteilte Zufallszahlen (z.B. mit `np.random.normal(...)`) benutzt werden?

Aufgabe 4 – Perlin Noise mit 3 Oktaven

Implementiere eine einfache Version von Perlin Noise mit 3 Oktaven.

Teil 1 – Grid erzeugen

Erzeuge zunächst ein regelmäßiges 2D-Gitter.

Beispiel:

8×8 Gridpunkte

Jeder Gridpunkt repräsentiert später einen zufälligen Gradientenvektor.

Teil 2 – Zufällige Gradientenvektoren

Generiere für jeden Gridpunkt einen zufälligen 2D-Vektor.

Beispiel:

$g=(x,y)$

Die Vektoren sollten normiert sein:

$|g|=1$

Teil 3 – Distanzvektoren berechnen

Für jeden Pixel:

1. Bestimme die umgebende Grid-Zelle.
2. Berechne die Distanzvektoren zu den vier Eckpunkten.

Beispiel:

$d = (p - \text{corner})$

Teil 4 – Skalarprodukte berechnen

Berechne pro Pixel für jede Ecke:

$g \cdot d$

also das Skalarprodukt aus:

- Gradientenvektor
- Distanzvektor

Dadurch entsteht der lokale Einfluss jedes Eckpunkts.

Teil 5 – Interpolation

(Pro Pixel:) Interpoliere zwischen den vier Skalarprodukten:

1. Verwende die Fade-Function $f(t) = 6t^5 - 15t^4 + 10t^3$ (Fade Function nach Ken Perlin) um sowohl den lokalen x-Wert als auch den lokalen y-Wert des Pixels zu glätten.
2. Interpoliere den lokalen geglätteten x-Wert horizontal zwischen dem Skalarprodukt des linken oberen Zellpunkt und dem Skalarprodukt des rechten oberen Zellpunkt
3. Interpoliere den lokalen geglätteten x-Wert horizontal zwischen dem Skalarprodukt des linken unteren Zellpunkt und dem Skalarprodukt des rechten unteren Zellpunkt
4. Anschließend interpoliere den lokalen y-Wert zwischen diesen beiden Ergebnissen

Die mathematische Formel für eine lineare Interpolation lautet: $\text{lerp}(a, b, \text{value}) = a + \text{value}(b - a)$ wobei a der Startwert und b der Endwert ist, und value der zu interpolierende Wert.

Teil 6 – Bild erzeugen

Berechne den Noise-Wert für jeden Pixel (Teil 3-5) und speichere das Ergebnis als Graustufenbild.

Teil 7 – Oktaven hinzufügen

Kombiniere anschließend mehrere Noise-Ebenen (=Graustufenbilder mit unterschiedlichen Frequenzen). Üblicherweise verdoppelt sich die Frequenz während die Amplitude sich halbiert.

Verwende mindestens 3 Oktaven.

Zusatzaufgabe

Färbe das Perlin Noise Bild:

Höhe	Farbe
niedrig	blau
mittel	grün
hoch	weiß

Aufgabe 5 – Mandelbrot-Menge

Implementiere die Mandelbrot-Menge.

Anforderungen

1. Visualisiere folgenden Bereich der komplexen Ebene: $x \in [-2.0, 1.0]$ $y \in [-1.5, 1.5]$
2. Verwende:
 - mindestens 100 Iterationen
3. Prüfe die Divergenzbedingung: $|z| > 2$
4. Färbe die Punkte abhängig von der Anzahl der Iterationen.

Hinweise

In python können komplexe Zahlen so beschrieben werden: `c = complex(real, imag)`

Zusatzaufgabe

Zoom in interessante Bereiche der Mandelbrotmenge hinein.

Bonusaufgabe – Generative Kunst

Erzeuge ein eigenes algorithmisches Kunstwerk.

Bedingungen

Verwende mindestens zwei der folgenden Konzepte:

- mathematische Funktionen
- Zufall
- Noise
- Rekursion
- Fraktale
- Farbfunktionen

Ziel

Experimentiere kreativ mit algorithmischer Bildgenerierung.